Day 8 exercises

Each exercise includes a problem statement, an example, and a **complete Java driver** that:

- contains an **empty logic method** (the one you must implement),
- **embeds all test cases** (including edge cases),
- creates any necessary temporary files internally,
- compares your output against the expected output **automatically**,
- prints **"All test cases passed"** or **"X test cases failed"**.

Everything is self-contained – just copy, implement the empty method, and run.

---

1. File Handling – Sum Numbers from File, Write Result to File

Topic: `BufferedReader`, `FileWriter`, basic file I/O.

Problem:
You are given the path of an **input file** that contains integers (one per line). Read all the numbers, compute their sum, and write that sum (as a string) into an **output file**. Return the sum.

Input:

- `inputFile`: path to a text file with one integer per line. File may be empty.

Output:

- The method writes the sum to `outputFile` and returns the sum.

Example:
Input file contents:

```
10
20
30
```

→ Sum = 60, output file contains `60`. Method returns `60`.

🔧 Driver Code (implement `sumNumbers`)

```
import java.io.*;
```

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class SumNumbersFile {
    public static int sumNumbers(String inputFile, String outputFile) throws IOException {
        // TODO: implement
        return 0;
    }

    public static void main(String[] args) throws IOException {
        // Each test case: {fileContent, expectedSum}
        Object[][] tests = {
            {"10\n20\n30\n", 60},
            {"", 0},
            {"-5\n15\n", 10},
            {"100", 100},
            {"0\n0\n0\n", 0}
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            String content = (String) tests[i][0];
            int expected = (int) tests[i][1];

            // create temp input file
            File inputFile = File.createTempFile("test_input_", ".txt");
            inputFile.deleteOnExit();
            Files.write(inputFile.toPath(), content.getBytes());

            // temp output file
            File outputFile = File.createTempFile("test_output_", ".txt");
            outputFile.deleteOnExit();

            int result = sumNumbers(inputFile.getAbsolutePath(), outputFile.getAbsolutePat

            // verify result
            if (result != expected) {
                System.out.println("Test " + (i + 1) + " FAILED (returned " + result + ",
                continue;
            }

            // verify output file content
            String written = new String(Files.readAllBytes(outputFile.toPath())).trim();
            if (!written.equals(String.valueOf(expected))) {
                System.out.println("Test " + (i + 1) + " FAILED (file contains \"" + writt
                continue;
            }

            passed++;
            System.out.println("Test " + (i + 1) + " passed");
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
```

```
}
```

---

2. BufferedReader – Count Words in a File

Topic: `BufferedReader`, tokenizing lines.

Problem:
Given a file path, read the file line by line using `BufferedReader`. Count the total number of **words** (separated by whitespace). Return the count.

Input:

- `filePath`: path to a text file.

Output:

- `int` – total word count. Empty file or file with only whitespace lines → 0.

Example:
File content:

```
hello world
java is fun
```

→ Word count = 4.

🔗 Driver Code (implement `countWords`)

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class WordCount {
    public static int countWords(String filePath) throws IOException {
        // TODO: implement using BufferedReader
        return 0;
    }

    public static void main(String[] args) throws IOException {
        Object[][] tests = {
            {"hello world\njava is fun\n", 4},
            {"", 0},
            {"   \n\t\n", 0},          // only whitespace
            {"singleword", 1},
```

```
            {"one two three four five", 5}
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            String content = (String) tests[i][0];
            int expected = (int) tests[i][1];

            File testFile = File.createTempFile("test_words_", ".txt");
            testFile.deleteOnExit();
            Files.write(testFile.toPath(), content.getBytes());

            int result = countWords(testFile.getAbsolutePath());
            if (result == expected) {
                passed++;
                System.out.println("Test " + (i + 1) + " passed");
            } else {
                System.out.println("Test " + (i + 1) + " FAILED (got " + result + ", expec
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

---

3. FileWriter – Write Lines then Read Back

Topic: `FileWriter` / `BufferedWriter` and reading.

Problem:
You are given a list of strings. Write each string **on a new line** into a file (using `FileWriter` or `BufferedWriter`).
After writing, read the file back and return the contents as a single string, **preserving the line breaks** (use `\n` to separate lines in the returned string).
If the list is empty, the file should be created but remain empty; return an empty string.

Input:

- `lines` : `List<String>`
- `filePath` : path where the file will be written.

Output:

- `String` with the exact file content (newline `\n` between lines, no trailing newline).

Example:
`lines = ["apple", "banana", "cherry"]` → file contains:

```
apple
banana
cherry
```

Return: `"apple\nbanana\ncherry"`

🔧 Driver Code (implement `writeThenRead`)

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class WriteThenRead {
    public static String writeThenRead(List<String> lines, String filePath) throws IOExcep
        // TODO: implement
        return "";
    }

    public static void main(String[] args) throws IOException {
        // test cases: {list of lines, expected output string}
        Object[][] tests = {
            {Arrays.asList("apple", "banana", "cherry"), "apple\nbanana\ncherry"},
            {Collections.emptyList(), ""},
            {Arrays.asList("hello"), "hello"},
            {Arrays.asList("line1", "", "line3"), "line1\n\nline3"},   // empty line in mi
            {Arrays.asList("special!@#", "chars$%^"), "special!@#\nchars$%^"}
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            @SuppressWarnings("unchecked")
            List<String> lines = (List<String>) tests[i][0];
            String expected = (String) tests[i][1];

            File testFile = File.createTempFile("test_fw_", ".txt");
            testFile.deleteOnExit();

            String result = writeThenRead(lines, testFile.getAbsolutePath());
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i + 1) + " passed");
            } else {
                System.out.println("Test " + (i + 1) + " FAILED");
                System.out.println("Expected: \"" + expected + "\"");
                System.out.println("Got:      \"" + result + "\"");
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
```

```
}
```

---

4. Serialization – Serialize and Deserialize a Person

Topic: `Serializable`, `ObjectOutputStream`, `ObjectInputStream`.

Problem:
Create a class `Person` with `String name` and `int age` that implements `Serializable`.
Implement a method that **deserializes** a `Person` object from a given file and returns it.
(The driver will first **serialize** a known `Person` to a temporary file, then call your method.)

Input:

- `filePath`: path to a file containing a serialized `Person` object.

Output:

- The deserialized `Person` object.

Example:
If the file contains a `Person("Alice", 30)`, the method returns that same object.

🔧 Driver Code (implement `deserializePerson`, ensure `Person` class is defined)

```
import java.io.*;

class Person implements Serializable {
    private static final long serialVersionUID = 1L;   // optional but good practice
    String name;
    int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Person person = (Person) o;
        return age == person.age && Objects.equals(name, person.name);
    }
}
```

```
public class DeserializePerson {
    public static Person deserializePerson(String filePath) throws IOException, ClassNotFo
        // TODO: implement (read object from file using ObjectInputStream)
        return null;
    }

    public static void main(String[] args) throws IOException, ClassNotFoundException {
        Object[][] tests = {
            {new Person("Alice", 30)},
            {new Person("Bob", 0)},
            {new Person("", 100)},           // empty name
            {new Person(null, 25)}           // null name allowed
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            Person original = (Person) tests[i][0];

            // create temp file and serialize the original person
            File tempFile = File.createTempFile("person_ser_", ".ser");
            tempFile.deleteOnExit();
            try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(temp
                oos.writeObject(original);
            }

            // call user's implementation
            Person result = deserializePerson(tempFile.getAbsolutePath());
            if (Objects.equals(original, result)) {   // safe for null fields
                passed++;
                System.out.println("Test " + (i + 1) + " passed");
            } else {
                System.out.println("Test " + (i + 1) + " FAILED");
                System.out.println("Expected: " + original.name + ", " + original.age);
                System.out.println("Got:      " + (result == null ? "null" : result.name +
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

---

5. Threads – Shared Counter with `synchronized` (extends `Thread`)

Topic: Extending `Thread`, synchronization.

Problem:
Create a `Counter` class with an `int count` and a `synchronized` method `increment()`.

Then create a custom thread class (extending `Thread`) that receives a `Counter` and a number of increments.
In `run()`, call `increment()` the given number of times.
Implement a method that creates the given number of threads, each performing `incrementsPerThread` increments on the **same** `Counter`, starts them all, waits for them to finish (using `join()`), and returns the final count.

Input:

- `numThreads`: number of threads (0 or more)
- `incrementsPerThread`: how many times each thread increments

Output:

- The final count (should be `numThreads * incrementsPerThread`).

Example:
`numThreads = 5, incrementsPerThread = 1000` → returns `5000`.

🔧 Driver Code (implement `runThreadsAndGetCount`)

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}

class IncrementThread extends Thread {
    private Counter counter;
    private int times;

    public IncrementThread(Counter counter, int times) {
        this.counter = counter;
        this.times = times;
    }

    @Override
    public void run() {
        // TODO: implement - call counter.increment() exactly 'times' times
    }
}

public class ThreadCounter {
```

```
public class ThreadCounter {
    public static int runThreadsAndGetCount(int numThreads, int incrementsPerThread) throw
        // TODO: create threads using IncrementThread, start them, join, return counter.ge
        return 0;
    }

    public static void main(String[] args) throws InterruptedException {
        int[][] tests = {
            {5, 1000},          // regular case
            {1, 1},             // minimal
            {0, 100},           // no threads → count 0
            {10, 0},            // 10 threads doing 0 increments → 0
            {3, 100000}         // large numbers to stress synchronization
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            int numThreads = tests[i][0];
            int inc = tests[i][1];
            int expected = numThreads * inc;

            int result = runThreadsAndGetCount(numThreads, inc);
            if (result == expected) {
                passed++;
                System.out.println("Test " + (i + 1) + " passed");
            } else {
                System.out.println("Test " + (i + 1) + " FAILED (got " + result + ", expec
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

---

6. Runnable – Shared Counter with `synchronized` (implements `Runnable`)

Topic: `Runnable`, `Thread`, synchronization.

Problem:
Same task as above, but this time implement the increment worker as a `Runnable`.
The method must create threads using `new Thread(runnable)`, run them concurrently, and return the final count. (Again, use a `synchronized` increment in the shared `Counter`.)

Input:

- `numThreads`, `incrementsPerThread`

Output:

- `numThreads * incrementsPerThread`

🔧 Driver Code (implement `runThreadsWithRunnable`)

```
class Counter {   // same synchronized Counter can be reused
    private int count = 0;
    public synchronized void increment() { count++; }
    public int getCount() { return count; }
}

class IncrementRunnable implements Runnable {
    private Counter counter;
    private int times;

    public IncrementRunnable(Counter counter, int times) {
        this.counter = counter;
        this.times = times;
    }

    @Override
    public void run() {
        // TODO: implement – call counter.increment() exactly 'times' times
    }
}

public class RunnableCounter {
    public static int runThreadsWithRunnable(int numThreads, int incrementsPerThread) thro
        // TODO: use IncrementRunnable, start threads, join, return counter.getCount()
        return 0;
    }

    public static void main(String[] args) throws InterruptedException {
        int[][] tests = {
            {5, 1000},
            {1, 1},
            {0, 100},
            {10, 0},
            {4, 250000}
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            int numThreads = tests[i][0];
            int inc = tests[i][1];
            int expected = numThreads * inc;

            int result = runThreadsWithRunnable(numThreads, inc);
            if (result == expected) {
                passed++;
                System.out.println("Test " + (i + 1) + " passed");
```

```
                System.out.println("Test " + (i + 1) + " passed");
            } else {
                System.out.println("Test " + (i + 1) + " FAILED (got " + result + ", expec
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

Summary of Topics Covered

#	Topic	Exercise
1	File Handling (BufferedReader, FileWriter)	Sum numbers from file, write sum back
2	BufferedReader	Count words in a file
3	FileWriter	Write list of lines, read back
4	Serialization	Serialize and deserialize a `Person` object
5	Threads + Synchronization	Shared counter with `Thread` subclasses
6	Runnable + Synchronization	Shared counter using `Runnable`